



TRUEFOUNDRY WHITEPAPER • ENTERPRISE
ARCHITECTURE & GOVERNANCE

Agentic
SDLC

The Agentic Software Factory, Explained: History, Architecture, and Enterprise Controls

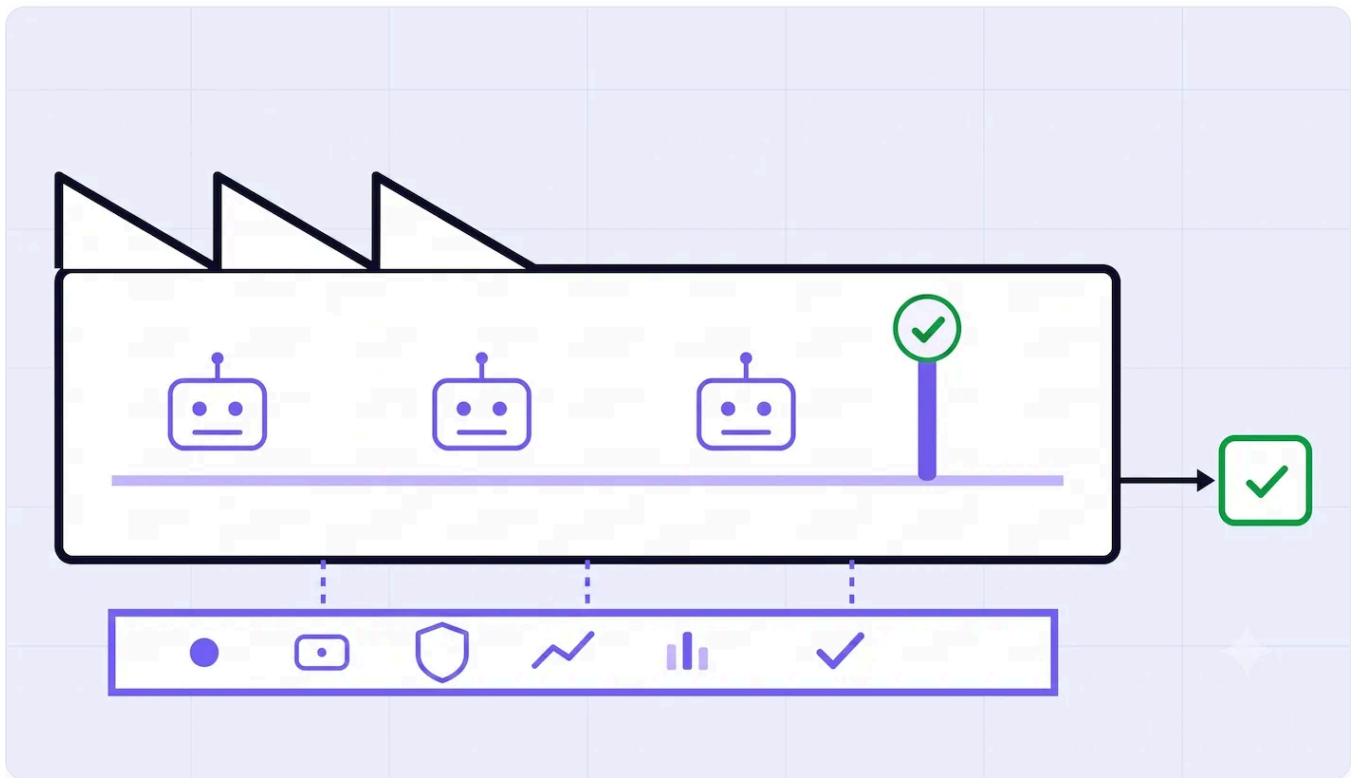


Boyu Wang

Technical Strategy & Architecture at TrueFoundry

Published: August 3, 2026

Category: Software Factory & AI Governance



The term of the season in software delivery is industrial: the **software factory** — increasingly the **agentic software factory** or **AI software factory** — a system that takes signals in one end (bug reports, specs, customer feedback, business requirements) and produces deployed, verified software out the other, with fleets of AI coding agents doing the work inside a feedback loop while engineers define intent and review results. In the first half of 2026, several organizations converged on an agent-native revival of the metaphor: BCG Platinion used "Agentic Software Factory" in March, and Factory.ai published its "Factory 2.0" manifesto in June — an interconnected, end-to-end system whose incremental units are agents, with the line we quote once: *"This system will be built by engineers, and in turn will build their software"* ([Factory.ai, June 2026](#)).

BCG Platinion declared the "Agentic Software Factory era" open, reporting that some organizations operating this model describe productivity gains of 3–5× — a reported practitioner estimate, not an independent category benchmark. [Spotify's official engineering account](#) reports more than 1,500 agent-generated pull requests merged into production, with 60–90% time savings for the migration workflows studied. And [OpenAI reported](#) building and operating an internal beta from an empty repository with three engineers

steering coding agents and no manually written code — the roughly one million lines including application code, tests, CI configuration, documentation, observability, and internal tooling (BCG Platinion, 2026).

The AI Engineer World's Fair put "enterprise software factories" among its five defining trends (AI Daily Brief), and practitioner definitional guides are now appearing weekly (murraycole.com; Plannotator). Plenty of those guides explain the assembly line. This one covers the part a manufacturing engineer would check first and the excitement mostly skips: the *utilities*. A physical factory is only as good as its power, plumbing, metering, and safety systems; an agentic software factory is only as good as its model supply, credential handling, quality control, and spend metering — the layer that isn't the agents and isn't the pipeline, and that this guide names, maps, and gives you a checklist for. As with all our young-term references, we will keep this page updated as the vocabulary and the practice evolve.

KEY TAKEAWAYS & CORE FINDINGS

- **A repeatable delivery system:** A software factory is a repeatable delivery system — standardized inputs, a defined assembly path, automated quality control, repeatable outputs — whose 2026 agent-native variant puts fleets of AI coding agents inside the pipeline as the workers, with engineers defining intent and reviewing evidence.
- **Deep historical lineage:** The idea predates AI by decades — Bemer's 1968 proposal, SDC's 1975 factory experiment, Microsoft's 2004 *Software Factories* book, and enterprise programs like BMW's; the 2026 development is an agent-native reinterpretation in which autonomous systems perform a larger share of the work inside the production system.
- **Heterogeneous velocity indicators:** Early company reports illustrate the potential velocity — BCG-reported 3–5x practitioner estimates, 60–90% savings on the migration workflows Spotify studied, an OpenAI internal beta of roughly a million lines from three engineers steering agents — but they are heterogeneous examples rather than a comparable benchmark set.
- **The utilities layer is foundational:** A factory is only as good as its utilities: model supply, credential handling, quality control, telemetry, and spend metering are the layer beneath the agents — and many factories benefit from shared model access, credential custody, policy, telemetry, and cost controls, although the mix varies by architecture and risk.
- **Mediated infrastructure boundaries:** Autonomous agents can increase credential, cost, and side-effect exposure. Controls are strongest when administrators enforce them at mediated infrastructure boundaries the agent cannot bypass; unmanaged paths and downstream systems still require their own protections.
- **Pragmatic evaluation before scaling:** Before scaling, evaluate model governance, credential custody, intermediate quality checks, traceability, cost controls, and human authorization in proportion to what the system can access and change.

What Is a Software Factory? A Definition

A software factory treats software delivery as a production system rather than a craft workshop: standardized inputs, a defined assembly path, intermediate quality controls, and repeatable, evidence-backed outputs. In an agentic software factory, coding agents perform a larger share of planning, implementation, testing, and review, while engineers define intent, establish acceptance criteria, govern access, and review evidence. The pattern can automate whole stations, assist humans within stations, or execute bounded subtasks. It is broader than a coding assistant because it includes the production system

around the agent, and broader than CI/CD because it governs work performed inside the delivery pipeline — not only the transitions between stages.

A Brief History: From Industrialized Reuse to Agent-Native Delivery

The term has a genuinely long pedigree, and tracing it explains what is and isn't new. The software-factory idea reaches back at least to Robert W. Bemer's 1968 proposal and System Development Corporation's published 1975 factory experiment — disciplined, repeatable software production on integrated tooling. Microsoft revived and productized a related model-driven formulation in its 2004 *Software Factories* book — templates, reference implementations, and guidance that automate accepted patterns — while later enterprise programs such as BMW's applied the name to standardized platforms, processes, and shared delivery infrastructure supporting more than 12,000 developers and roughly 140,000 CI jobs daily. The 2026 development is not the invention of the software factory, but a renewed agent-native interpretation in which autonomous coding systems perform a larger share of the work inside the production system — changing throughput, review economics, failure modes, identity requirements, and the relationship between production controls and autonomous execution.

Software Factory vs Coding Agents, Loop Engineering, and Platform Engineering

Three boundaries locate the term among its neighbors:

Versus a Coding Agent

The agent is a **worker**; the factory is the **production system** the worker operates in — inputs, stations, QC, evidence. A brilliant agent in an undefined process is a craftsman, not a factory, which is one reason reported outcomes with similar agents vary so widely across organizations.

Versus Loop Engineering

Loop engineering is the craft of **one autonomous verify-and-retry cycle**: its triggers, checks, budgets, and stopping conditions. The factory is loops at *industrial organization*: many loops, staffed stations, shared QC, one flow from signal to ship.

Versus Platform Engineering

Platform engineering's golden paths and self-service infrastructure standardized the environment humans worked in. The factory inherits it and intensifies a critical question: what happens when the entity using the golden path is an **autonomous agent** with credentials, a 24/7 duty cycle, and high variability?

The Factory Synthesis

The answer to scaling autonomous agents safely across enterprise delivery is **not more agents and not a better pipeline**. It is the **utilities layer**: power, plumbing, metering, and safety interlocks that govern execution across all stations.

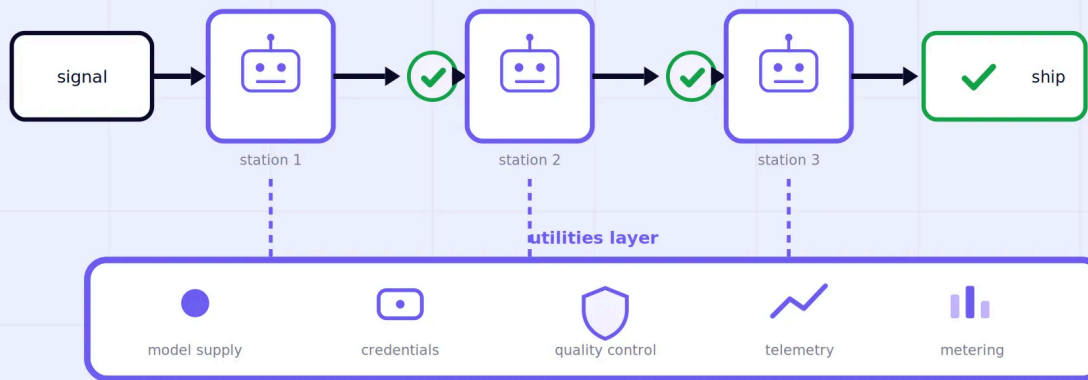


Figure 1: The agentic software factory's shape — signals in, agent-staffed stations along a defined path, QC gates, evidence-backed output — with the utilities layer beneath: model supply, credentials, quality control, telemetry, and metering. (TrueFoundry editorial synthesis; original graphic).

The Software Factory in the Enterprise: The Utilities Layer

TrueFoundry's role is the utilities layer, scoped honestly to what its products mediate. Participating MCP tool calls routed through the [MCP Gateway](#) can use centralized server registration, authentication, authorization, guardrails, and observability — while direct APIs, browser access, local tools, shell operations, CI systems, and other bypass paths require separate controls. For registered MCP servers, credentials can be centrally managed and injected rather than embedded in agent definitions; configured OAuth flows and auth overrides can preserve user-specific access where supported — TrueFoundry supports current MCP authentication patterns including centrally managed credentials, per-user OAuth, token passthrough, and configured auth overrides; public documentation currently presents differing availability signals for first-class Agent Identity and OBO delegation, so confirm support for the target deployment and version before relying on complete user→agent delegation-chain attribution.

Configured gateway policies can apply guardrails to matching LLM and MCP operations; repository tests, static analysis, CI checks, sandbox and egress controls, merge policy, and connected evaluators remain separate parts of the quality system. Managed Agent Harness runs can provide execution traces, and gateway records capture participating model and MCP operations. Enforced AI Gateway budget rules can block additional matching LLM requests after configured limits are exceeded; warn-only rules provide visibility — compute, CI infrastructure, tool costs, and downstream side effects require separate limits.

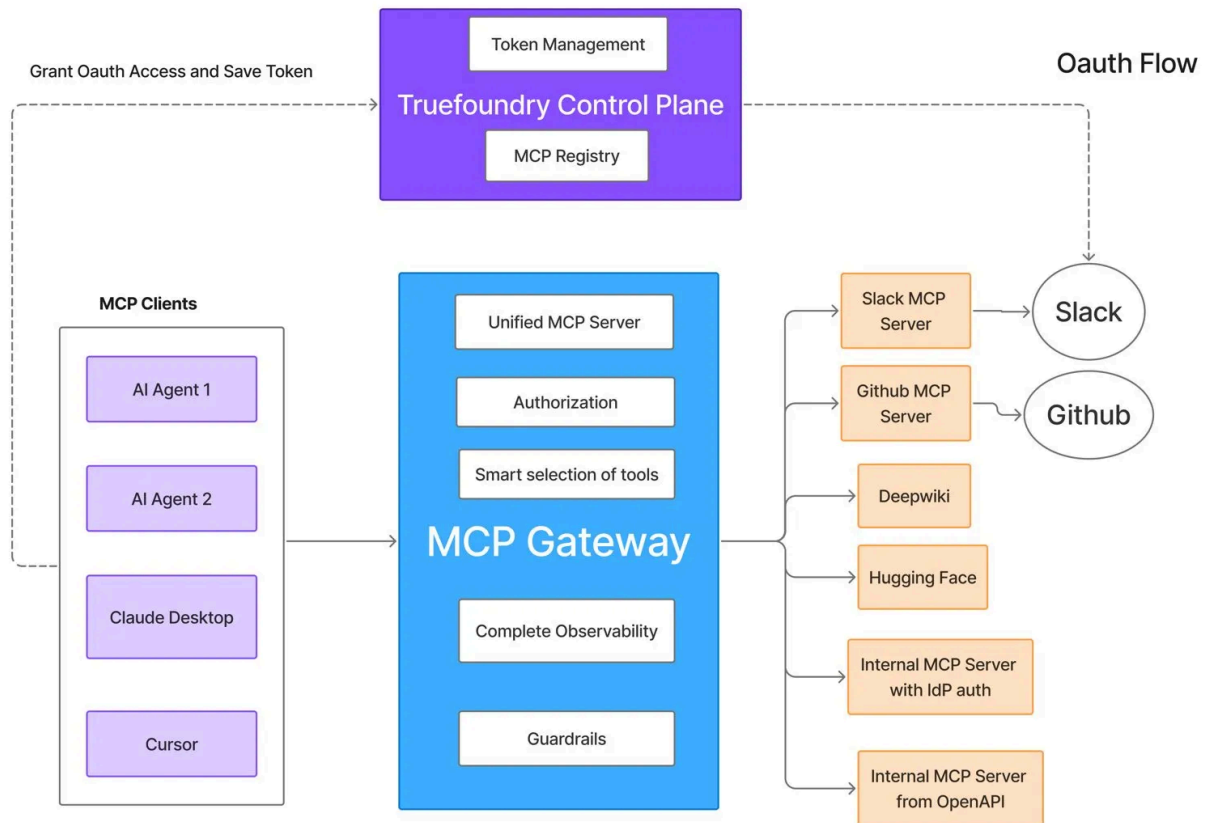
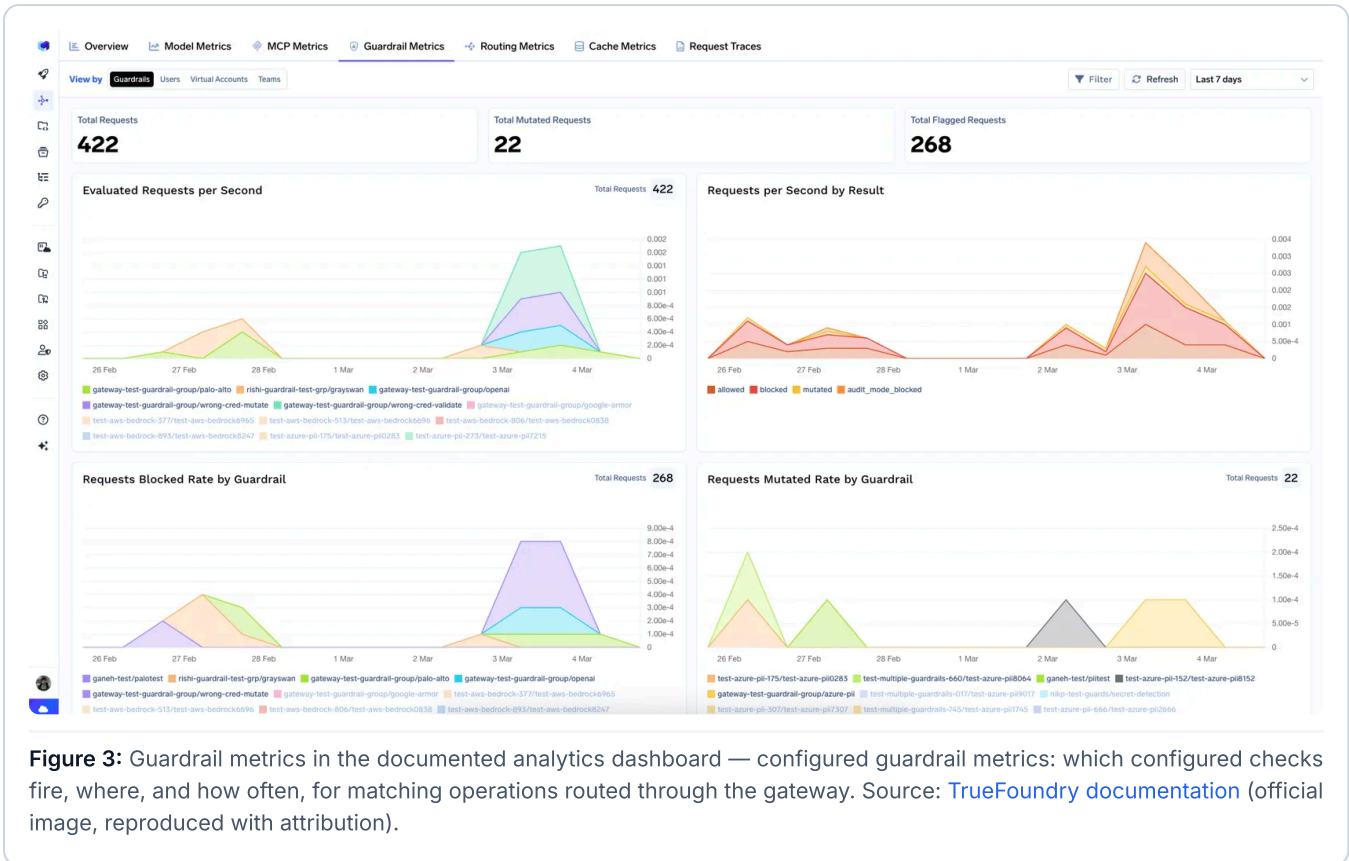


Figure 2: TrueFoundry MCP Gateway architecture as documented — the factory's credential plumbing: participating MCP tool calls routed through the MCP Gateway can use centralized authentication, access controls, guardrails, and observability — while direct APIs, browser access, local tools, queues, and other bypass paths require separate controls using centrally managed or user-specific credentials where configured instead of embedded secrets. Source: [TrueFoundry documentation](#) (official diagram, reproduced with attribution).

THE FACTORY INSPECTOR'S QUESTION

In the system's worst hour: which operations are mediated, which identities and permissions apply, what limits are enforced, what evidence survives, and where must a human authorize continuation? A credible answer normally spans the agent runtime, model gateway, MCP gateway, sandbox and network controls, source-control and CI policy, and the downstream systems the agents can change.



Enterprise Controls to Evaluate Before Scaling

Before deploying fleets of autonomous agents into production codebases and environments, engineering leaders must audit their infrastructure against seven fundamental control dimensions:

KEY GOVERNANCE & CONTROL AUDIT CHECKLIST

- **Model Mediation & Gateway Routing:** Which model calls are mediated by the AI Gateway, and which bypass it directly to external APIs?
- **Credential Custody & Delegation:** Where are credentials stored, injected, delegated, and refreshed? Are secrets embedded in agent configs or injected dynamically via an authenticated MCP gateway?
- **Action Modality Classification:** Which actions are MCP calls, direct APIs, browser actions, shell operations, or CI jobs? What boundary isolates each?
- **Quality Verification Multi-Tiering:** Which quality checks are gateway guardrails, repository tests, evaluators, static linters, or merge policies?
- **Spend Metering & Quotas:** Which costs are limited by the AI Gateway (tokens/prompts), and which require infrastructure, cloud compute, or vendor controls?
- **Human-in-the-Loop Interlocks:** Which tool calls can Agent Harness approvals pause — and which approvals belong to source control (PR reviews), CI/CD gates, or downstream applications?
- **Risk-Dependent Boundaries:** What can the factory build, and what can it break? Are limits proportionate to blast radius?

The Future of the Software Factory: What We'll Be Watching

Predictions, lightly held. Watch whether organizations standardize on a common category name; whether factory platforms remain coding-specific; how repository and CI controls integrate with agent runtimes; how firms measure defect escape, review burden, cost, and rollback rates; whether vendor-neutral model and tool interfaces become normal; and how responsibility divides between factory vendors, internal developer platforms, and governance infrastructure, the same migration we watched with loop engineering and agentic engineering.

Whether the factory pattern extends beyond code is worth watching: the pattern — standardized signal in, agent-staffed stations, QC, evidence-backed artifact out — describes document production, data pipeline maintenance, and security operations just as well, and "factory" may end up naming the general industrial form of agentic work, with the utilities recurring — with variation — across them.

Expect the economics to force the utilities conversation upward: at Spotify's reported scale (more than 1,500 merged agent-generated PRs to date), organizations may need to treat token spend, credential exposure, and auditability as explicit operational concerns, and whether "show me the factory's meters and interlocks" becomes a standard procurement question is worth watching. Consolidation pressure between factory vendors and the platform layer is plausible, and vendor-neutral utilities may reduce switching costs and make it easier to change agent or model suppliers as capabilities evolve: a factory whose power, plumbing, and meters are your own plane can swap its robots as the frontier moves — which is the model-independence argument Factory.ai itself makes, carried to its infrastructure conclusion. We will keep this guide updated as the practice matures; one hypothesis worth testing is whether durable advantage accrues less from agent count than from the quality of the production system around them.

Software Factory FAQ

Q: What is a software factory in AI?

An agentic (AI) software factory is a repeatable delivery system in which fleets of AI coding agents execute standardized, multi-step work — planning, coding, testing, review — along a defined pipeline with automated quality control, producing evidence-backed outputs like pull requests with passing checks, while engineers define intent, set quality bars, and review results.

Q: How is a software factory different from using coding agents?

A coding agent is one worker or execution component; a software factory is the broader production system around it — standardized inputs, a defined assembly path, quality controls, telemetry, and metering. Enterprise adoption therefore depends on both agent capability and the workflow, controls, and evidence surrounding it.

Q: Is "software factory" a new term?

No. The software-factory idea reaches back at least to Bemer's 1968 proposal and SDC's 1975 experiment, was reformulated in Microsoft's 2004 *Software Factories* book, and was later applied to enterprise delivery platforms such as BMW's. What changed in the 2026 agent-native revival is the division of labor: autonomous coding systems perform a larger share of planning, implementation, testing, and review, changing throughput, oversight economics, failure modes, and identity requirements.

Q: What infrastructure does an enterprise software factory need?

Enterprise controls vary by architecture and risk. Common needs include governed model access, centrally managed credentials for participating tools, repository and CI checks, configured gateway guardrails, runtime and request evidence, separate limits for LLM and infrastructure spending, and human authorization around consequential actions. No single platform necessarily supplies or mediates every layer.

References & Primary Sources

- **Primary case sources:** [Spotify Engineering, "1,500+ PRs Later" \(background coding agent, Part 1\)](#) (merged-PR count; 60–90% savings on studied migration workflows); [OpenAI, "Harness Engineering"](#) (internal beta, team size of 3 engineers, repository contents, zero manually written code constraint).
- **Historical lineage:** [IEEE Computer Society, Robert W. Bemer](#) (1968 software-factory proposal); [System Development Corporation's published 1975 "Software Factory" work \(IEEE\)](#); [Greenfield, Short, Cook & Kent, *Software Factories* \(Wiley, 2004\)](#); [AWS case study, BMW Software Factory](#) (12,000-developer, ~140,000 daily CI-job scale).
- **The term and its moment:** [Factory.ai, "Factory 2.0: From coding agents to software factories"](#) (June 2026: agent-native definition, model independence); [BCG Platinion, "The Agentic Software Factory"](#) (era framing; 3–5x gains; Spotify/OpenAI figures); [AI Daily Brief on the AI Engineer World's Fair](#) (enterprise software factories as defining trend); definitional guides: [Factory.ai guide](#), [Plannotator](#) (2006 Microsoft & BMW lineage), [murraycole.com](#) (practitioner build view; secrets and sandboxes).
- **TrueFoundry technical documentation:** [Virtual Models](#); [MCP Gateway \(Official Architecture\)](#); [Guardrails Overview](#); [Budget Limiting v2](#); [Analytics & Guardrail Metrics](#).
- **Related industry literature:** [Loop Engineering at Enterprise Grade](#); [Operating Loops at Fleet Scale](#); [The Agentic Token Explosion in CI/CD](#); [Running Coding Agents in the Enterprise](#); [Spec-Driven Development](#); [OAuth at the MCP Layer](#); [Graph Engineering Explained](#).

One direct quotation under fifteen words is used, from the cited Factory.ai announcement, attributed inline; all other definitions, figures, and practitioner statements — including the BCG productivity and Spotify/OpenAI numbers, which are those organizations' reported claims — are paraphrased with links to the originals. "Factory" is Factory.ai's product and brand; "software factory" is used here as the common industry term the cited sources establish, and this document makes no claims about any vendor's product. The utilities framing and the enterprise mapping are editorial onto capabilities paraphrased from public documentation current at the time of writing; the product images are documentation assets reproduced with attribution. None of the cited companies or authors evaluates or endorses TrueFoundry. This is a living reference and will be revised as the practice and vocabulary evolve.